late

Transport Layer

Now we are going onto our next layer: Transport layer.

Its services & protocols:

It provides logical communication between application processes running on different hosts.

Transport protocols actions in end systems e.g PCs:

- Sender: breaks app messages into segment. & pass to network layer
- receiver: reassemble segments to messages. & pass to app layer.

PDU: segment.

BHATTI

CamScanner

y/date

2 transport layer protocols available:

1) TCP
2) UDP

Understanding difference in transport layer & network layer protocols.

Network layer: logical communication between hosts (and devices).

Transport layer: logical communication between processes.

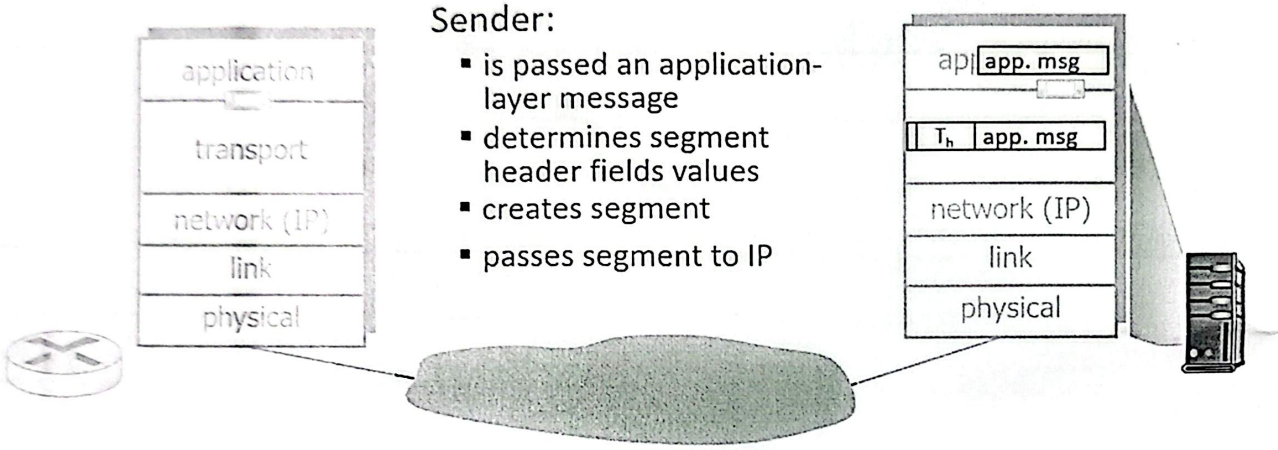
remember processes run inside hosts.

BHATTI

CamScanner

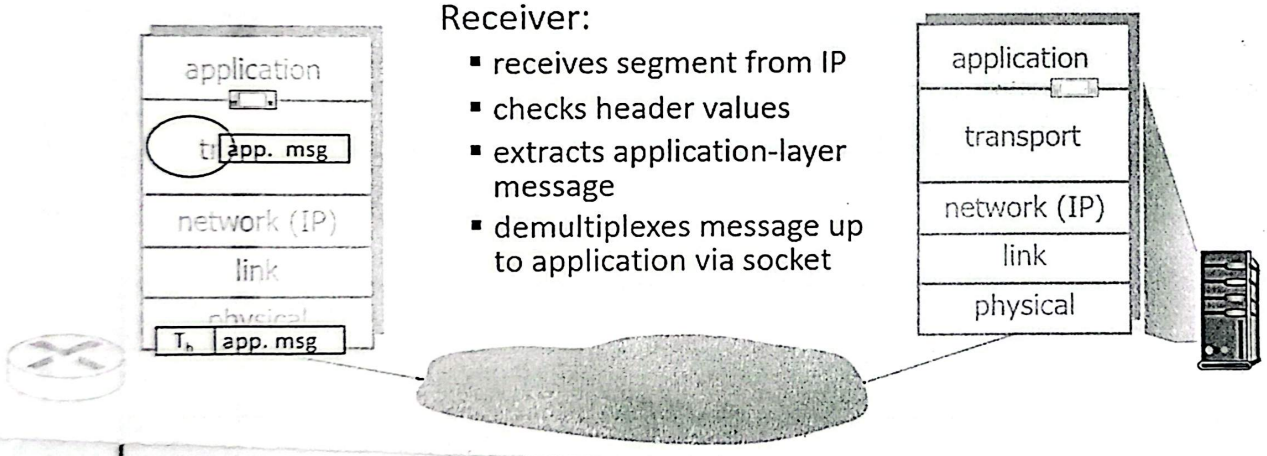
day/date

Transport layer Actions



Sender:

- is passed an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP



Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket

'date

2 principal Internet transport protocols

TCP: Transmission control Protocol

- reliable-, in-order delivery
- congestion control
- flow control
- connection setup.

UDP: User datagram protocol:

- unreliable, unordered delivery
- no-frills extension of best-effort IP.

BHATTI

CamScanner

lay/date

Multiplexing & demultiplexing

At transport layer, the job is to deliver data not just to the host device but to the specific application socket running on that host.

Multiplexing: (at sender host): Gathering data chunks from multiple application sockets, encapsulating each chunk with transport header fields (such as source / destination ports) to create segments & passing to network layer

Demultiplexing: (At receiver host): Examine header fields of an incoming transport-layer segment to identify the correct receiving socket & deliver the segments payload to that exact socket. So remove $H(h_t)$ & send the message to correct socket.

BHATTI

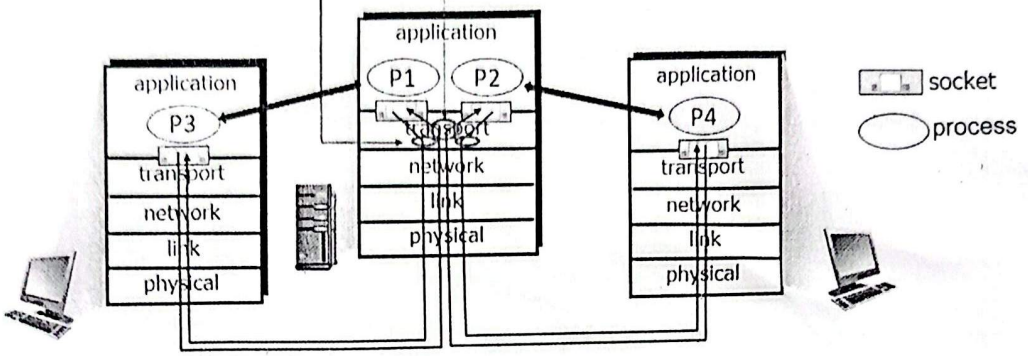
CamScanner

multiplexing at sender:

handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing at receiver:

use header info to deliver received segments to correct socket



Port numbers & Socket Addressing:

Every transport layer segment contains 2 vital fields in its header:

- Source Port number (16 bits).
- Destination ports number (16 bits).

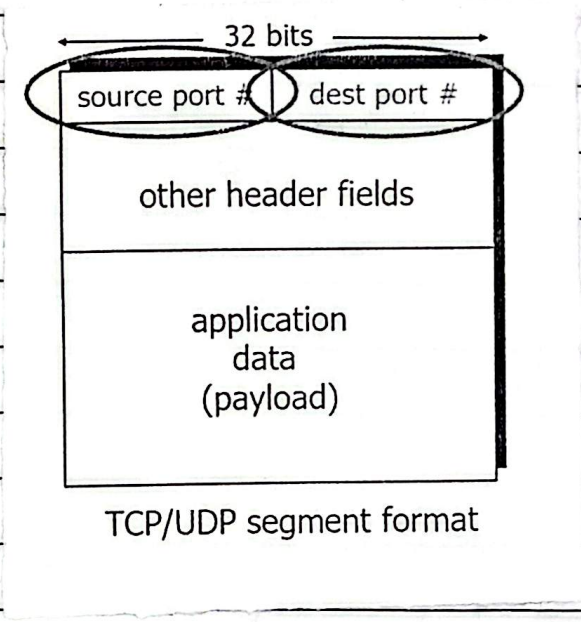
- Port numbers range from 0 – 65535 as $2^{16} - 1 = 65535$.

BHATTI

day/date

well-know ports: (0-1023): Reserved for standard protocols like Http, DNS, SMTP etc.

Dynamic ports (1024-65535): Automatically assigned by host OS when a client app opens a socket.



TCP/UDP segment format

* host gets Port number from transport header, segments. But the IP address comes from network layer header & its PDU: datagram. Each datagram has a segment which has a Port.

By using IP + port, Host can direct segment to correct socket.

BHATTI

CamScanner

day/date

How demultiplexing work?

- host receives IP datagrams
  - each datagram has source IP address, destination IP address
  - each datagram carries one transport-layer segment
  - each segment has source, destination port number
- host uses *IP addresses & port numbers* to direct segment to appropriate socket

* remember datagram are from network layer. we need IP for identifying host system & port to identify particular process.

There are 2 ways of demultiplexing depending upon UDP or TCP:

1) Connectionless Demultiplexing (UDP)
2) Connection-oriented demultiplexing. (TCP)

BHATTI

CamScanner

day/date

1) Connection-less demultiplexing: (UDP):

This is for UDP sockets.

In this, when making socket, host local port number must be specified.

It is identified by a 2-tyle:

UDP socket identifier = (Destination IP address, Destination Port number)

UDP socket identifier should be added by the sender so that it can be sent to the correct device & process.

How it works?

- When Host A creates a UDP segment, it includes a source port & destination port.

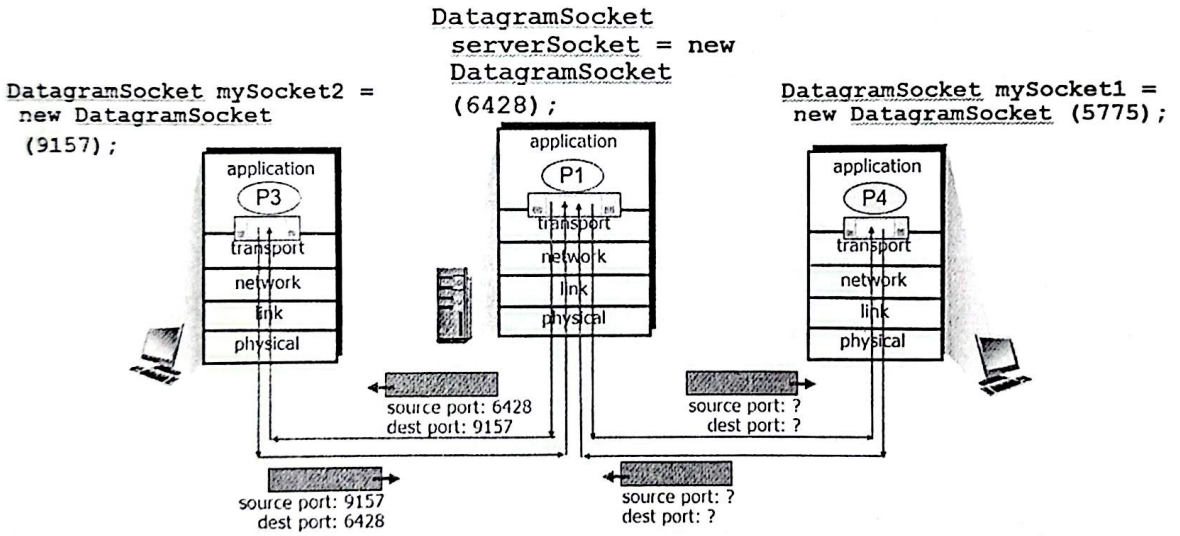
BHATTI

CamScanner

day/date

- When Host B receives a UDP segment, its transport layer checks only the Destination Port number & destination IP. to direct segment to Socket.

- IMP: If 2 different client hosts send UDP segments to the same destination IP & port e.g DNS server on port 53, both segments will be directed to same socket regardless of source IP's or ports.



BHATTI

CamScanner

day/date

2) Connection-oriented multiplexing (TCP):

This is for TCP sockets.

A TCP socket is identified by a 4-Tuple:

TCP socket identifier = { Source IP, Source Port, Destination IP, Destination Port }

How it works?

- When a client connects to a TCP server (e.g. a web server on port 80), server accepts connection & creates a new, dedicated socket bound to that 4-tuple.

~~socket~~

IMP: even if 100 different client browsers send HTTP GET requests to Port 80 of same server:

BHATTI

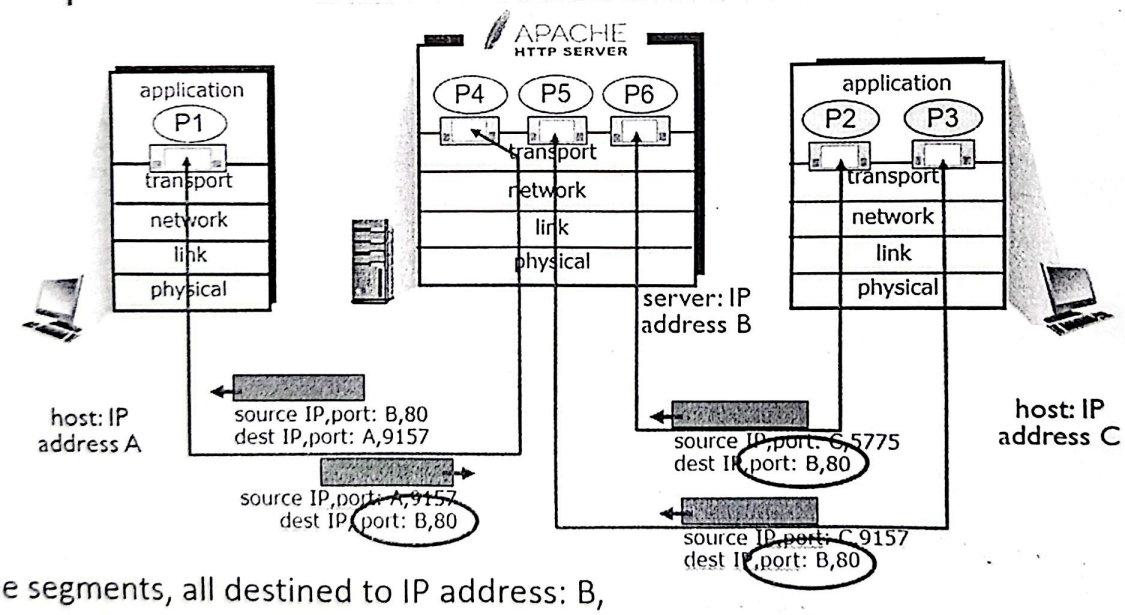
CamScanner

day/date

- each client has a different (source IP, source port)

- The server uses the full 4-tuple to demultiplex each packet to its own dedicated connection socket.

- This allows TCP to maintain independent state, sequence numbers, flow control & timers for every connection.



Three segments, all destined to IP address: B, dest port: 80 are demultiplexed to *different* sockets

CamScanner

day/date

UDP vs TCP in connection-Oriented or not:

Feature	UDP (Connectionless)	TCP (Connection-Oriented)
Socket Identifier	**2-Tuple** (Dest IP, Dest Port)	**4-Tuple** (Src IP, Src Port, Dest IP, Dest Port)
Server Sockets	Single socket handles requests from all clients	One welcoming socket (listen), plus one dedicated connection socket per active client
Demultiplexing Logic	Checks destination port only	Checks all 4 fields to match exact connection state
Header Overhead	Minimal (8 bytes)	Larger (20 bytes minimum)

BHATTI

CamScanner

day/date

UDP : User Datagram protocols:

It is a minimal, lightweigh protocol.
It is described as no-frills because it almost adds nothing to the underlying Network Layer (IP).

- It is connectionless:
  - no handshake needed
  - each UDP segment handled independently of others.

- Unreliable Delivery:
  - Delivery is not guaranteed
  - No flow or congestion control.

- Low overhead:
  A UDP header is only 8 bytes long compared to TCP's ≥ 26 Byte header.

BHATTI

CamScanner

day/date

UDP is used in:

- streaming
- DNS
- SNMP
- HTTP/3.

IF reliable transfer needed over UDP:

- add reliability at app layer
- add congestion control at app layer.

e.g HTTP/3, QUIC.

UDP sender / receiver actions:

Sender actions:

- Is passed an app layer message
- determines UDP header fields & values.
- creates UDP segment
- passes segment to IP (network layer)

receiver actions

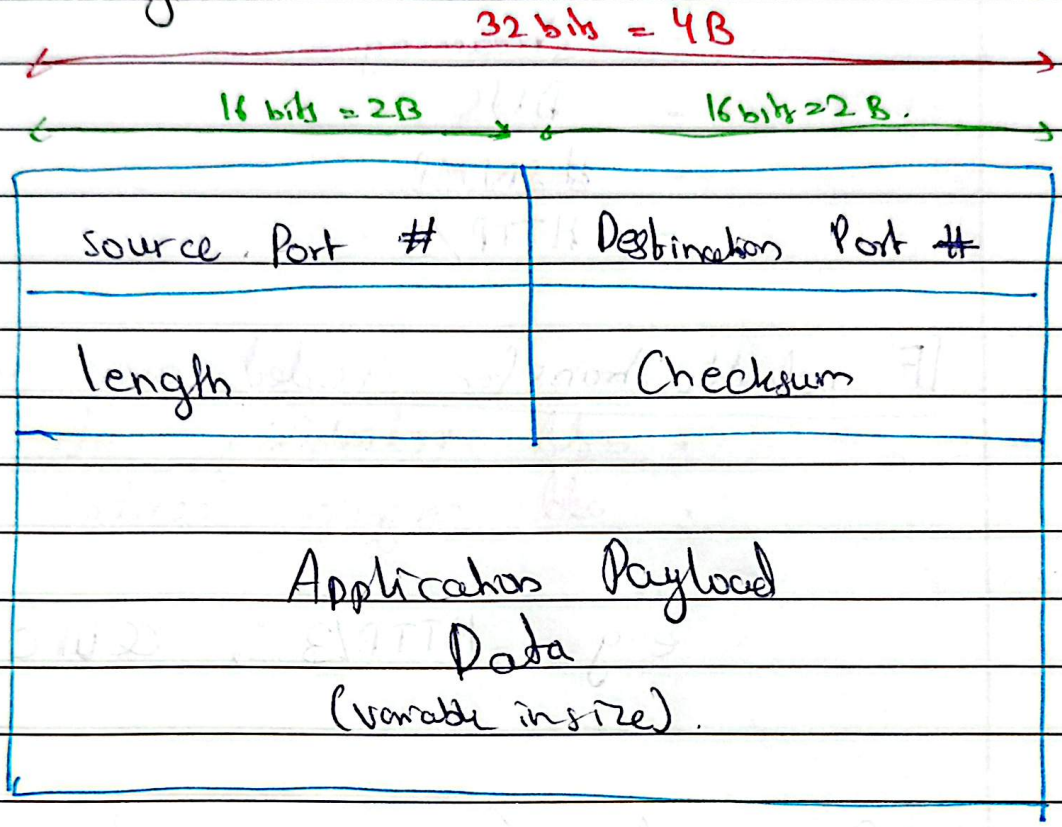
- receives segment from IP (network layer).
- checks UDP checksum
- extracts APP-layer message
- demultiplexes message up to app via Socket.

BHATTI

CamScanner

lay/date

UDP segment header:



- Source Port # (16 bits) : identify sending app's ports on source host
- Destination Port # (16 bits): identify receiving app's port on destination host
- length (16 bits): Specifies the length of UDP segment in Bytes. (header + Data). Minimum value can be 8 Bytes (an empty packet with only header).

BHATTY

day/date

- Check sum (16 bits): Provides error detection for segment

UDP Check sum:

This is used to check error (i.e flipped bits) in transmitted segment.

How sender Calculates the Checksum:

1) Treats the entire UDP segment (header + payload) as a sequence of 16 bit Integeres.

2) SUM all 16-bit integers together using 1's compliment arthemetic

3) Takes the 1's compliment flip all 0s to 1s & 1s to 0s of the final sum.

4) Places this result into UDP checksum field.

BHATTI

CamScanner

day/date

How receiver verifies the checksum.

1) Add up all 16 bits integers (including checksum itself) using 1's complement arthemetic.

2) Result check:

If all bits are one's (11111111...) , no error.

If there is even one zero , error

How do bits all become one?

From sender.

Checksum = 1's complement of (Sum of Data words)

in receiver:

Total sum = Checksum + Sum of data words.

BHATTI

CamScanner

day/date

Total sum = 1's complement of (Sum of Data) + sum of Data.

which should always lead to G(1111111--) in no error.

Check sum example:

1) Consider we have 2 16-bits things in UDP header:

1 with value = 58982 , 2 with value = 54613

now calculate checksum:

```
                ①   ①   ①   ①
58982 =     1 110 0 110 0 110 0 110
54613 =  ①  1 101 0 101 0 101 0 101
         +  ---------------------------
           1 011 1 011 1 011 1 011 1 0
           +                       1
           ---------------------------
           1 011 10 11   10 1111 00
           ↓ inverted (1's complement)
Check sum = 0 100 01 00   01 0000 11
```

to remember to wrap your carry & add it in 1's complement addition

BHATTI

day/date

To find check sum:

1. Standard binary addition
2. Wrap around the carryout if generated & add it back.
3. Take 1's complement.

BHATTI

CamScanner

day/date

Checksum is weak protection:

- Checksum is actually a poor way of protection.

because e.g we are checking

$5 + 4 = 9$

(5 + 4: header; 9: check sum)

but

$6 + 3$
$3 + 6$
$4 + 5$
$2 + 7$

} all equal to 9.

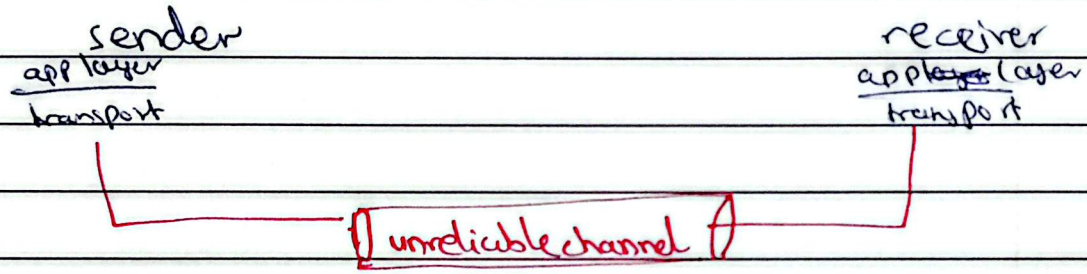
so if we flip all header's some bits we might be able to pass check sum test

BHATTI

CamScanner

Transport layer (Principles of reliable data transfer)

UDP is a good way to transfer data but it is unreliable



we should know that all of the internet is unreliable channels.

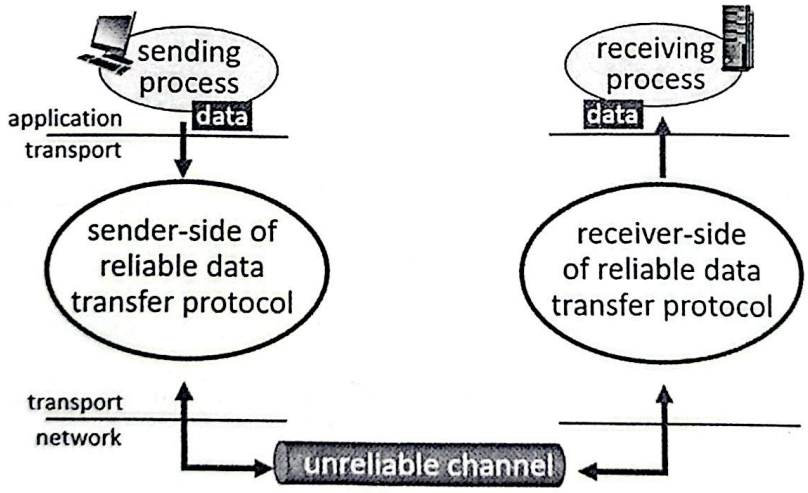
we are talking about whole internet. not like an ethernet wire connecting 2 PC's only which would actually make it reliable.

BHATTI

day/date

- our channel always remains unreliable, so how do we ensure reliable data transfer?

A way is to implement reliable data transfer (rdt) protocols on both sender & receiver sides.



BHATTI

CamScanner

/date

- with this we also need to see that we cannot make our channel reliable so our protocols should be so good that unreliable channel doesn't matter.

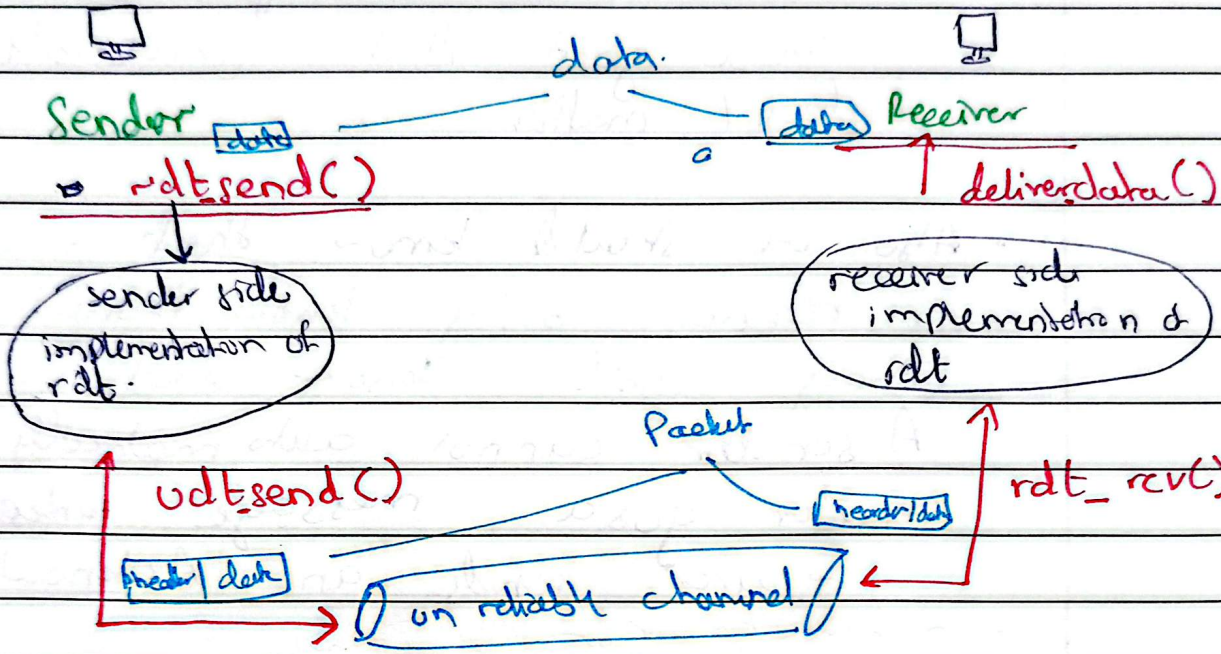
- Also we should know that, sender & receiver only know themselves. They don't know each other. A sender cannot automatically see receiver got a message unless receiver sends an Acknowledgement.

BHATTI

CamScanner

ate

Understanding commands of interfaces in rdt:



rdt_send (data): This is called by applayer for passed data to deliver to ~~receiver applayer~~ transport layer protocol.

It has rdt in name because it is currently inside the device which is reliable.

, Sender side

BHATTI

(Packet)

udt_send(): This is called by the RDT protocol layer / transport layer.
It transmits the packet over the unreliable channel.
it has udt (unreliable dt) bcz it is sending over unreliable channel.

- Sender side.

~~rdt_received~~

rdt_rcv (packet): Called by a lower layer e.g network layer when a packet arrives & it is used to signal the transport layer that a packet has arrived.

After this the RDT layer/transport layer extracts the packets & does its functions like error handling & removing header.

- Receiver side.

BHATTI

date

deliver_data (dcetcp): called by transport layer to deliver the data to app layer.
- Receiver side

There are also many more helper functions involved but we will do them one by one in next topic where we discuss different rdt's.

~~But the rdt protocol will be~~

~~we will discuss~~

BHATTI

/date

rdt protocols:

multiple rdb protocols exists which evolved over time:

1) rdt 1.0
2) rdt 2.0
3) rdt 2.1
4) rdb 2.2
5) rdt 3.0

we will use Automata's i-finite state Machines to learn them & also timing diagrams at end.

1) rdt 1.0:

" reliable transfer over a reliable channel"

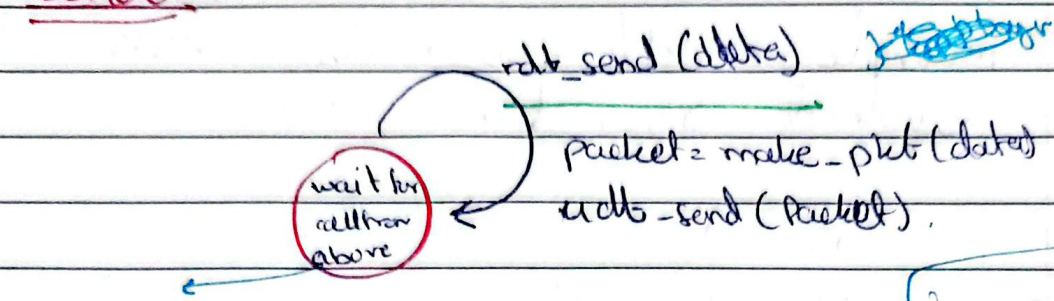
In this we consider the channel is perfectly reliable:
- no bit errors
- no loss of packets.

In this both sender & receiver are very simple & have only one state each.

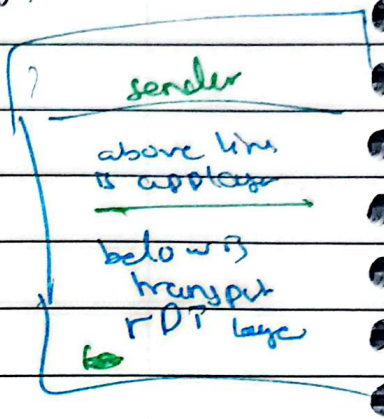
BHATTI

CamScanner

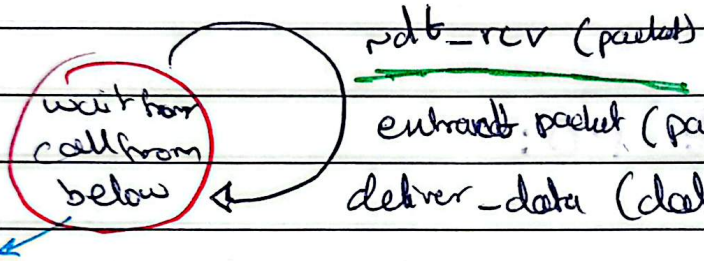
Sender:



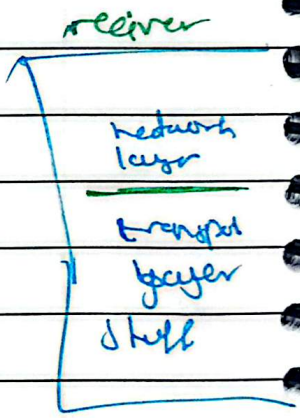
above means app layer.



receiver



means network layer or channel.



New functions:

make_pkt (data): returns data with header added, which is a packet.

extract packet (packet, data): extracts the packet content into the data buffer.

BHATTI

//date

IMP:

- Data flows unidirectional only (sender to receiver). Acknowledgements are unnecessary because it is reliable.

BHATTI

CamScanner

day/date

2) rdt 2.0

"channel with bit errors"

now we consider that our channel also has bit errors

* We are not considering packet loss currently.

Bit can flip i.e our data can be corrupted but not lost on this channel.

How are bit error handled ?.

1) Error Detection: Checksum.

2) Receiver Feedback:

If data is not corrupted then receiver sends: ACK, control message or feedback which means Positive acknowledgement

If data is corrupted then NAK is sent telling sender data was corrupted.

BHATTI

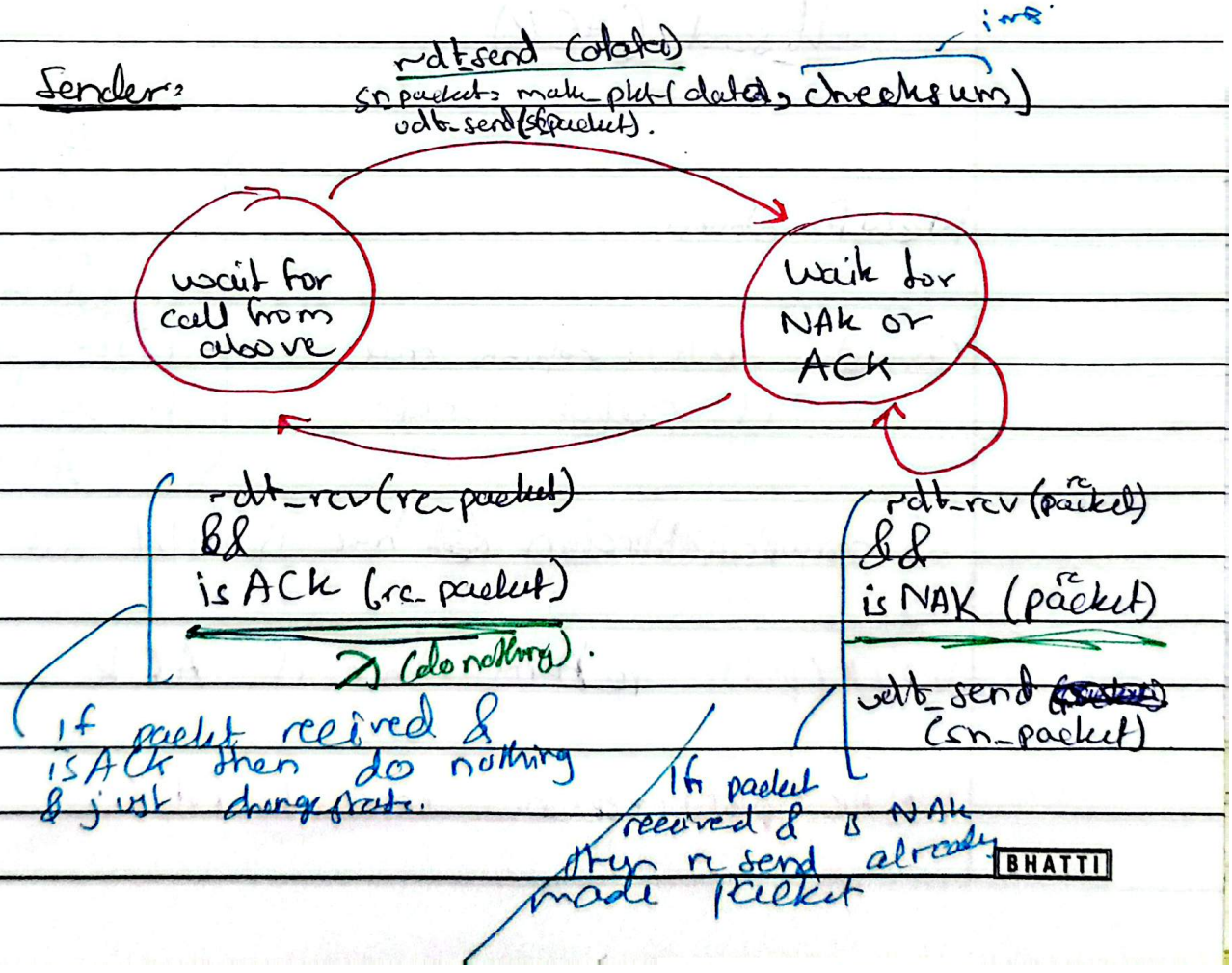
CamScanner

day/date

3) Retransmission: If sender receives NAk then it retransmits the same packet

Also here it follows Stop & wait method.
where sender sends the data & waits for receiver for response.

State machines:



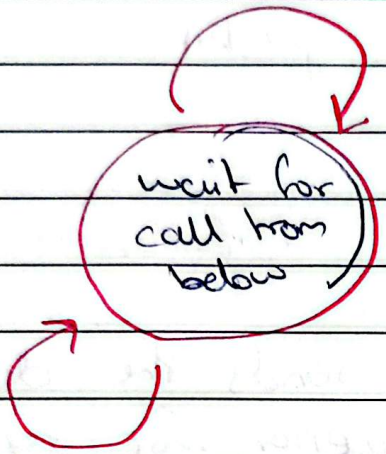
BHATTI

day/date

Receiver

rdt_rcv(rc_packet) && corrupt(rc_packet)
udt_send (NAK)

If packet received is corrupt send NAK. (no need to make NAK's packet).



rdt_rcv(rc_packet) && NOTcorrupt(rc_packet)
extract (rc_packet, data)
deliver_data(data)
udt_send (ACK)

If packet received is not corrupt send to upp layer either extract & send the sender ACK.

New functions:

Corrupt(pkt): returns true if packet corrupt.

not corrupt(pkt): returns true if packet not corrupt.

IsACK(pkt): returns true if ACK

IsNAK(pkt): returns true if NAK.

BHATTI

CamScanner

/date

Fatal flaw in RDT2.0:

What if ACK/NAK become corrupted?

- Sender doesn't know what happened in receiver side.
- Sender might just retransmit if corrupted ACK or NAK but then duplicates need to be handled because receiver doesn't know if it is a new packet or duplicate

BHATTI

CamScanner

lay/date

3) rdt 2.1:

"Handling corrupted ACKs & NAKs"

rdt 2.1 solves the fatal flaw in rdt 2.0 by introducing sequence numbers.

only one-bit (1 or 0) sequence numbers are enough because sftf rdt 2.1 follows stop & wait so mathematically it is enough.

How it works?

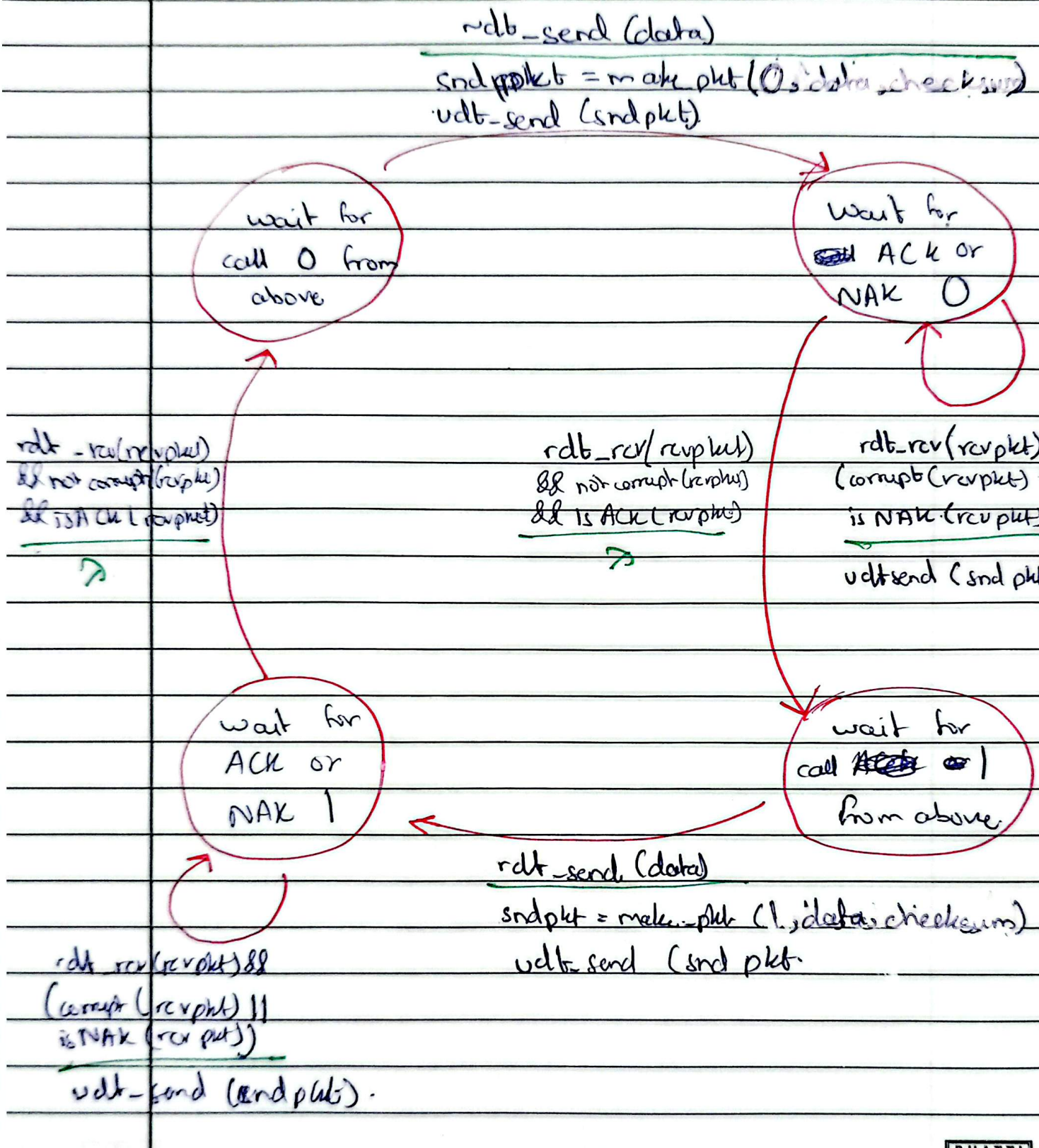
If an ACK or NAK get corrupt, then sender will retransmit but receiver will now see that new retransmission is with the wrong sequence number so it rejects that & but still sends ACK to sender so that sender doesn't keeps on retransmitting the wrong packet.

BHATTI

CamScanner

day/date

Sender FSM:



BHATTI

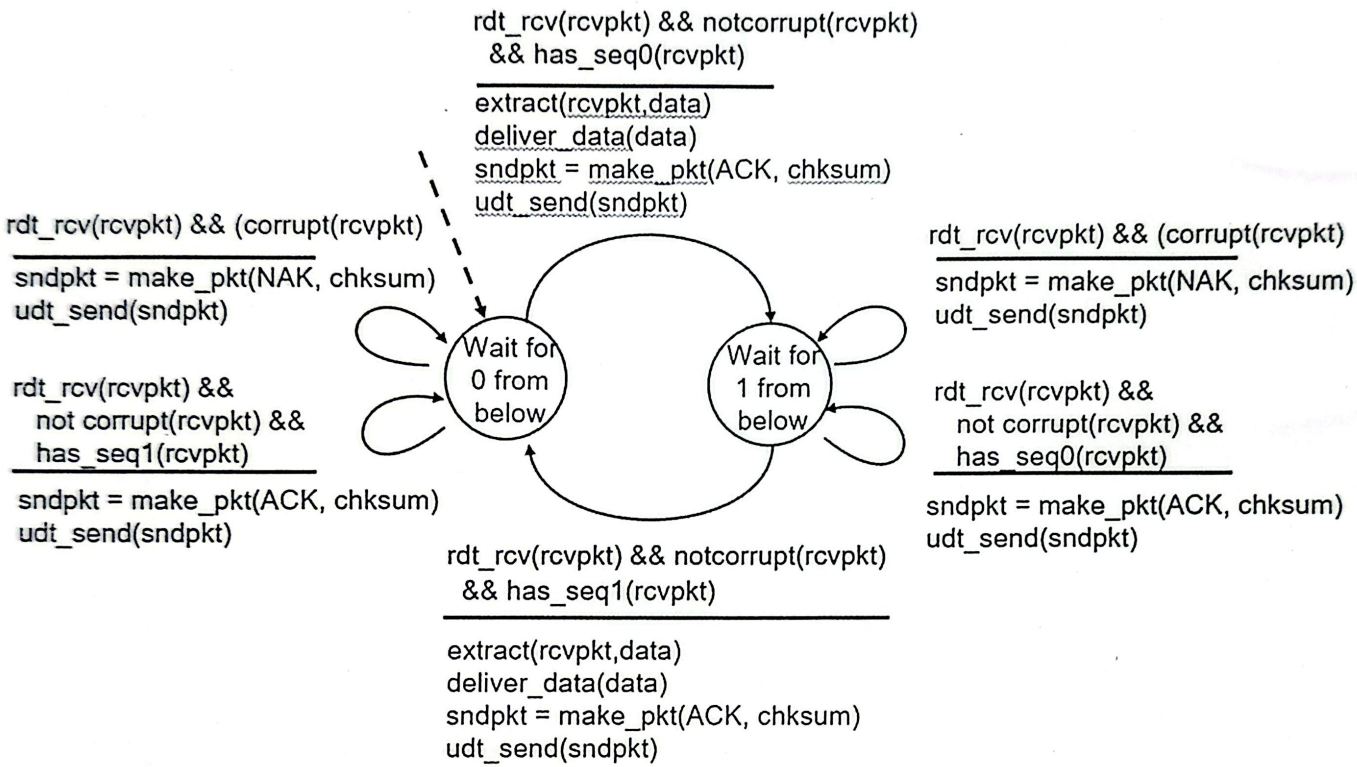
day/date

Sender Side (4 States):

1. **Wait for call 0 from above:** Upper layer gives data → sender creates `make_pkt(0, data, checksum)` , transmits it, and moves to *Wait for ACK/NAK 0.*
2. **Wait for ACK or NAK 0:**
   - Corrupted feedback OR `isNAK` : Retransmits packet 0, stays in *Wait for ACK/NAK 0.*
   - `notcorrupt` AND `isACK` : Moves to *Wait for call 1 from above.*
3. **Wait for call 1 from above:** Upper layer gives data → sender creates `make_pkt(1, data, checksum)` , transmits it, and moves to *Wait for ACK/NAK 1.*
4. **Wait for ACK or NAK 1:**
   - Corrupted feedback OR `isNAK` : Retransmits packet 1, stays in *Wait for ACK/NAK 1.*
   - `notcorrupt` AND `isACK` : Moves back to *Wait for call 0 from above.*

CamScanner

Receiver FSM:



CamScanner

day/date

Imp:

- now for sending ACK or NAK we are making proper packets because for their corruption check we need checksum.

- If on a state a packet is not corrupt but wrong sequence number then we stay on same state but send ACK, this tells that sender that receiver has already received this packet long ago, so no need to send duplicate & go to next packet.

- If packet is corrupt then no need to check sequence number, bcz packet needs to be demanded again by sending NAK.

BHATTI

CamScanner

day/date

New functions:

has seq 0 : checks if sequence number in packet is 0.

has seq 1: checks if sequence number in packet is 1.

IMP:

- only 0 & 1 are sufficient enough for rdt 2.1 because it uses stop & wait protocol.

BHATTI

CamScanner

y/date

b) rdt 2.2:

"NAK Free protocol"

This protocol ~~receiver~~ ~~the~~ achieves same reliability as rdt2.1 but it eliminates NAK's completely.

How does it do?

now our receiver will send (ACK, 0) or (ACK, 1).

If the receiver wants a Packet 0 but get corrupted packet it sends (ACK, 1), acknowledges the last good packet, this will make the sender retransmit the data.

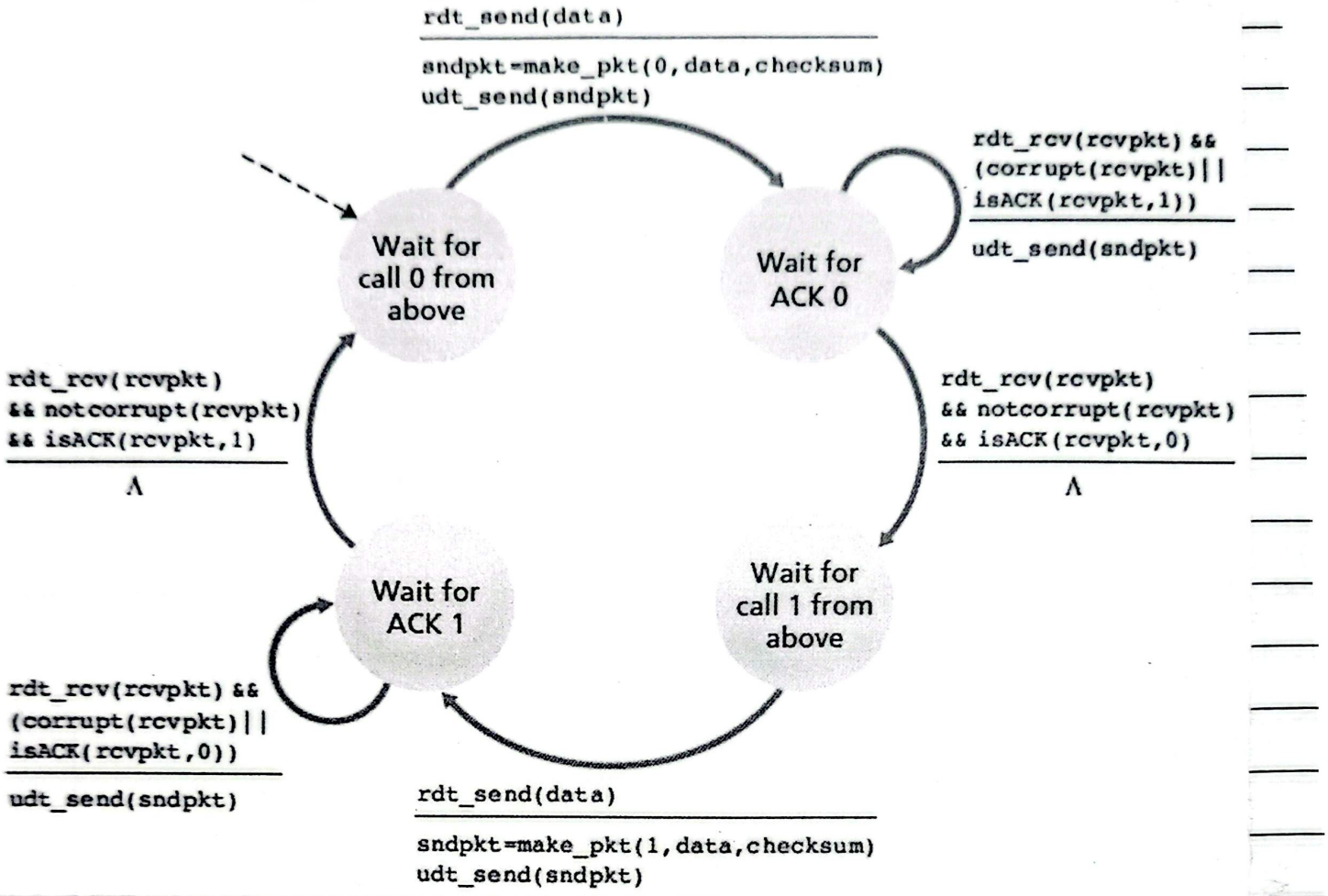
so now we don't even need NAK any more.

BHATTI

CamScanner

day/date

Sender diagram:

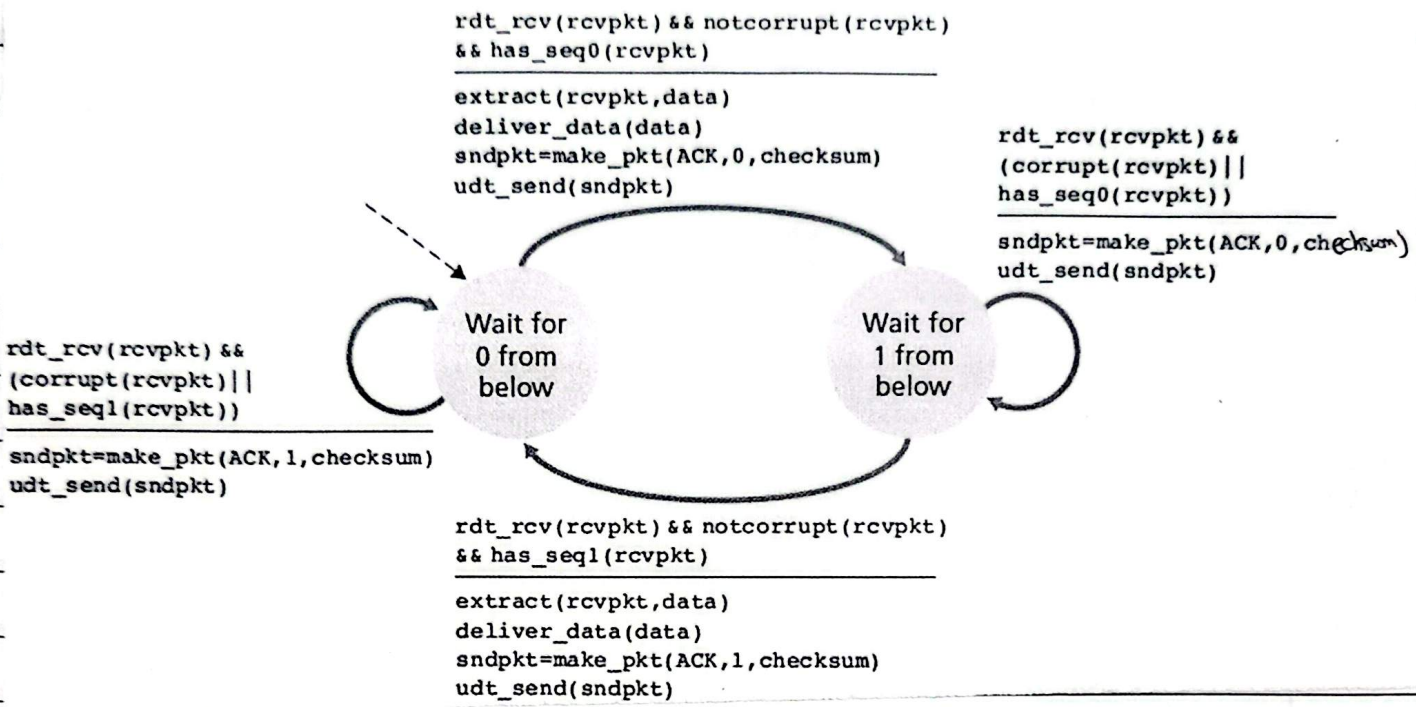


IMP:

- If on waiting for ACK 0 & we get ACK 1 then we stay on same state while resending package.

day/date

Receiver side:



IMP:

- Now if we get wrong packet or corrupted packet we simply send ACK with previous sequence Number.

- Now our ACK packet:

sndpkt = make_pkt(ACK, 1, checksum)

extra added bit here

BHATTI

CamScanner

y/date

IMP:

- This way we are able to completely remove NAK & this is how TCP even works.

- still rdt 2.2 follows the stop & wait so only one bit sequence is enough.

BHATTI

CamScanner

day/date

5) rdt 3.0

"Channels with bit errors & packet loss"

- we have solved problem of bit errors but what if our channel sometimes gets packet loss. meaning receiver never receives it e.g due to some disconnection for some time.

To handle this, a Countdown Timer is used at the sender.

Sender waits a reasonable amount of time for ACK & retransmits if no ACK received..

BHATTI

CamScanner

/date

How it works?

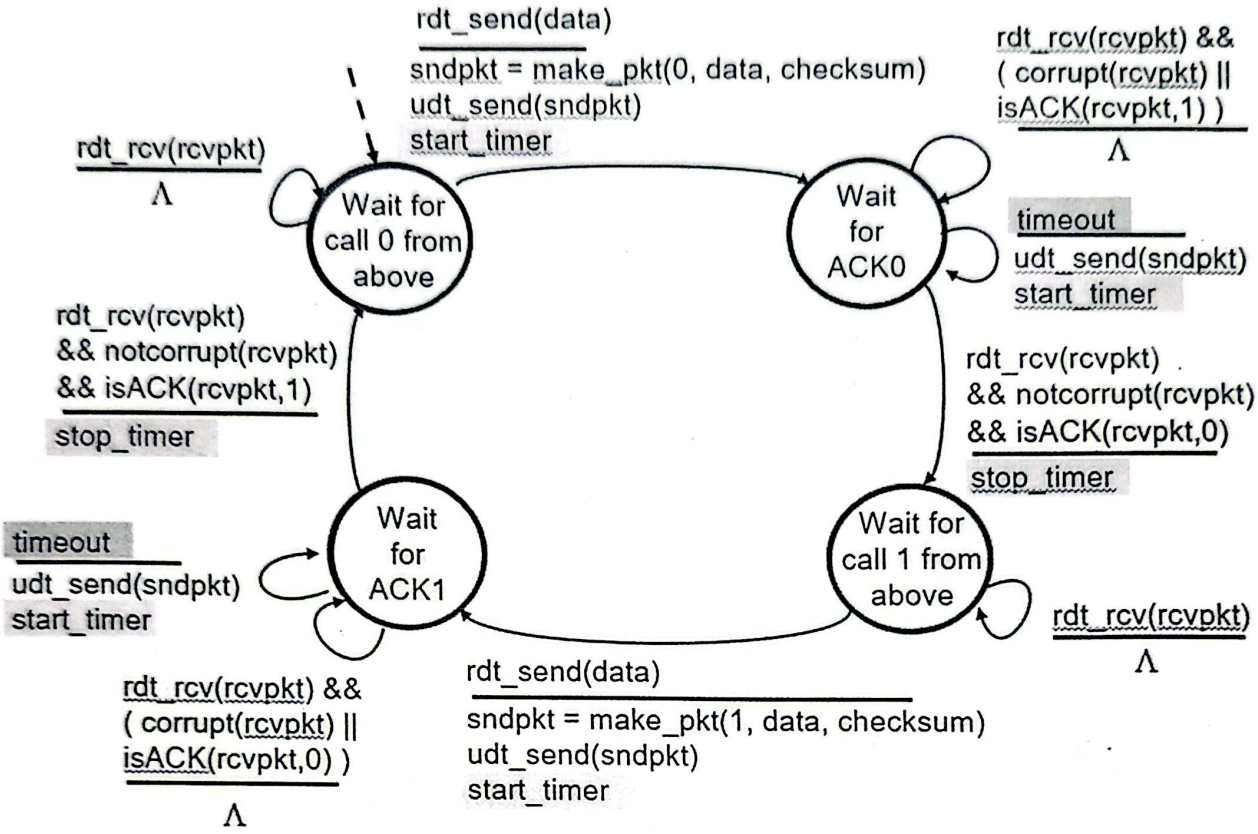
- Sender starts a timer when packet is transmitted
- waits for an ACK.
- If an ACK arrives before end of time then stop timer & advance to next state
- If no ACK arrives until end of time then assume packet loss & retransmit the packet & restart timer.

A problem comes that what if it is a long RTT & the timer expires prematurely. So for this a duplicate will be send & sequence number (1 or 0) will be used to handle that.

BHATTI

day/date

Sender FSM:

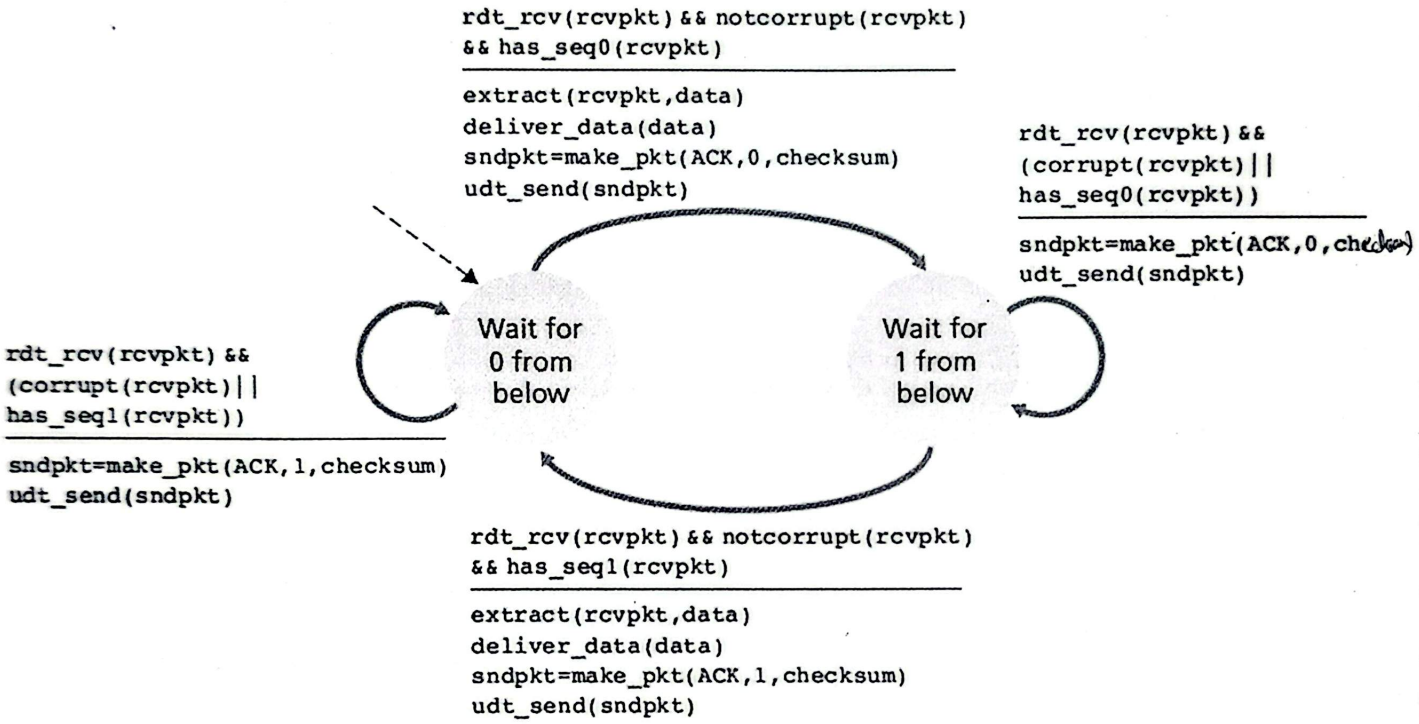


Imp:

- Same as rdt 2.2 (no NAK) but conditions of time out added.

Receiver FSM:

Exactly same as rdt 2.2 because timer is added in sender & receiver remains unchanged.



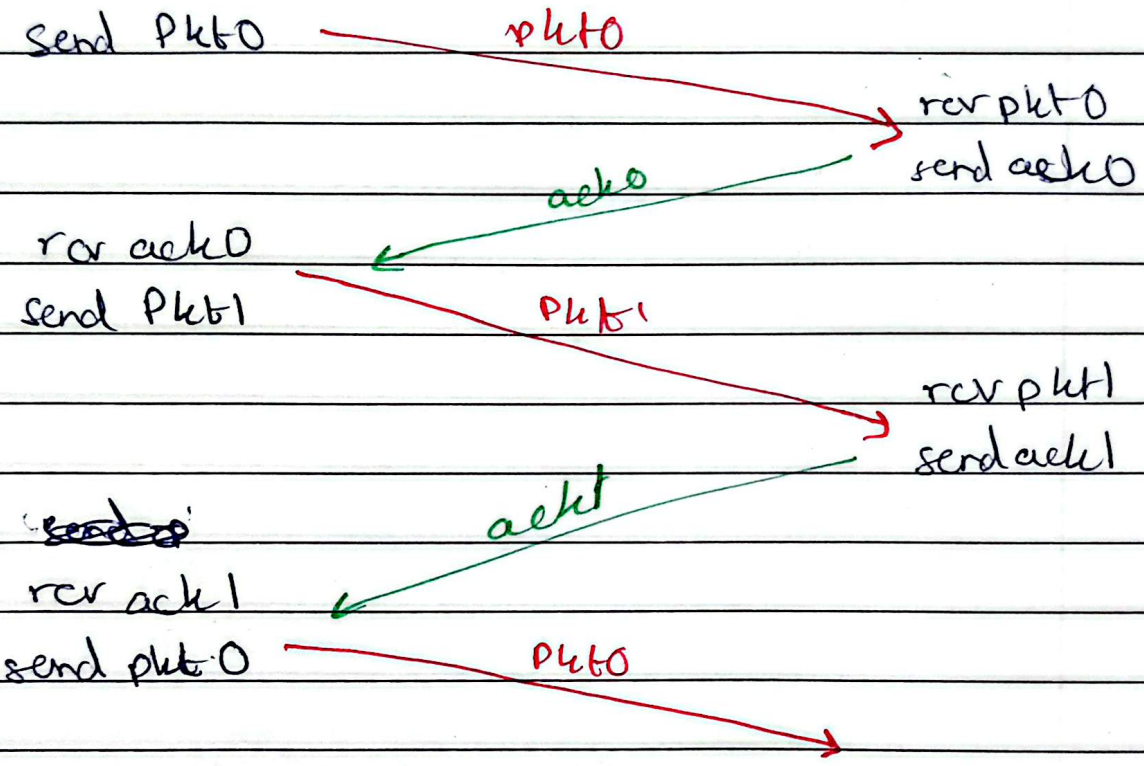
CamScanner

ay/date

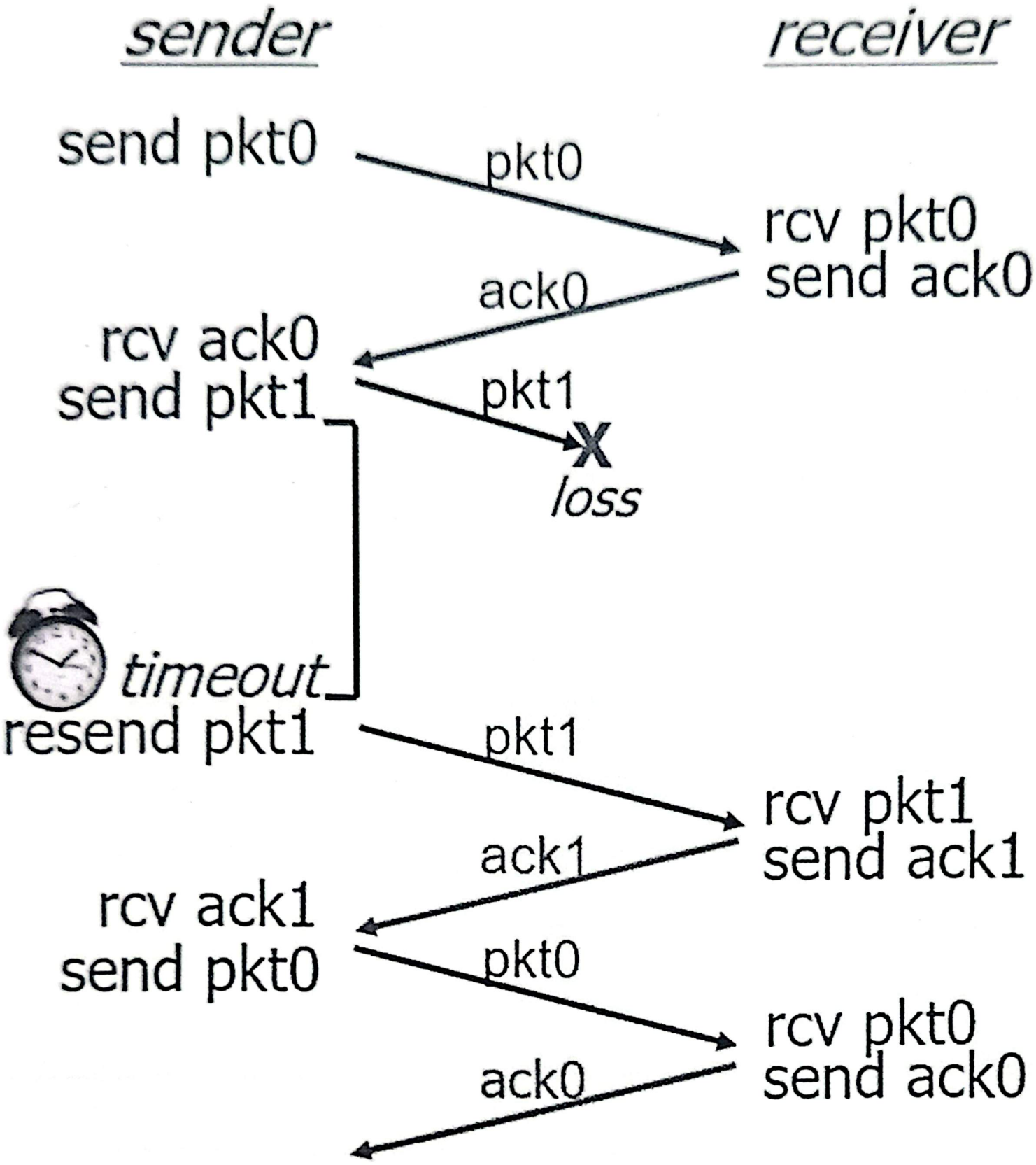
rdt 3.0 in action:

we will use timeline diagrams now.

1) no loss:

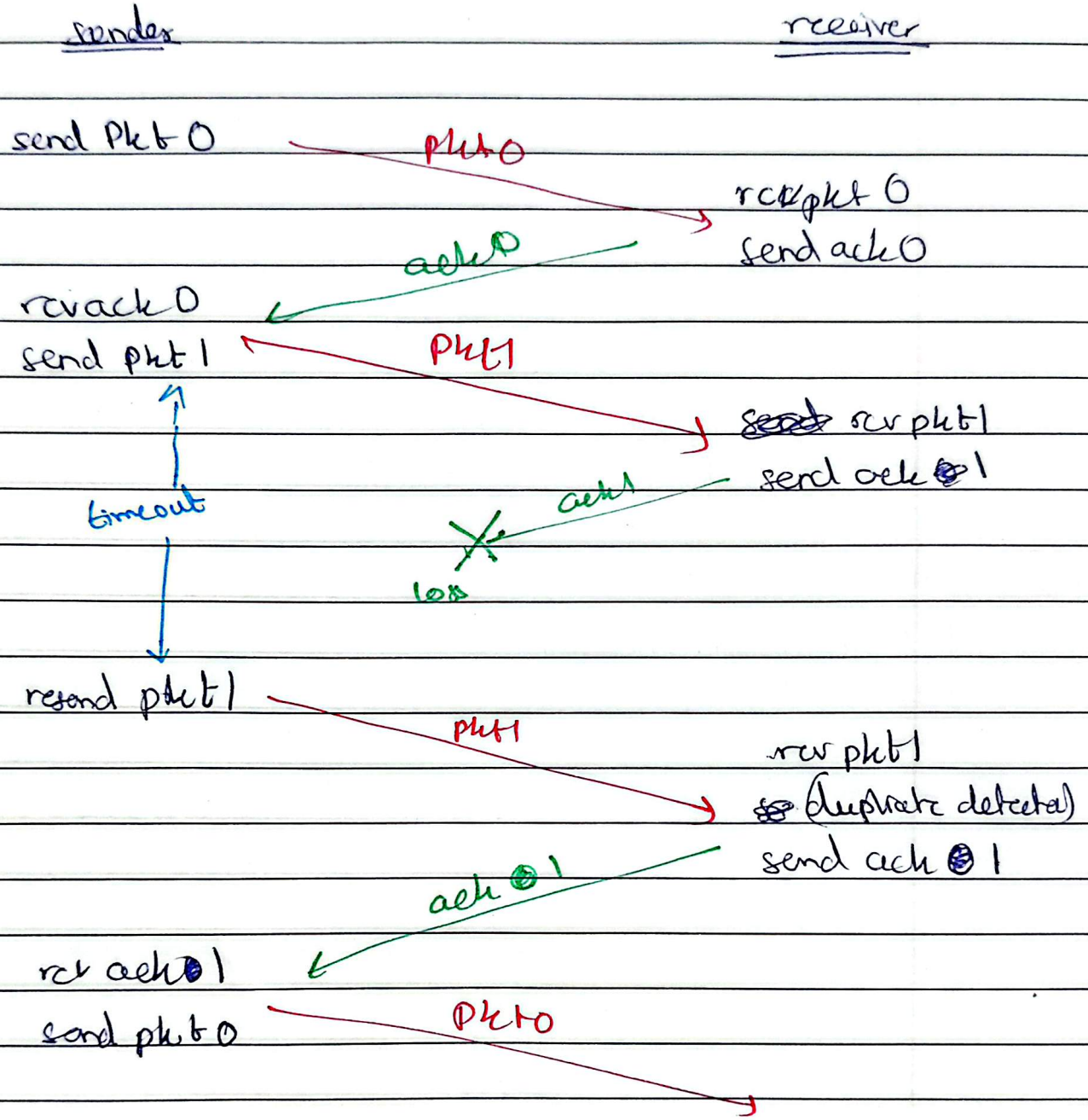


BHATTI


sender
receiver
send pkt0
pkt0
rcv pkt0
send ack0
ack0
rcv ack0
send pkt1
pkt1
X
loss
timeout
resend pkt1
pkt1
rcv pkt1
send ack1
ack1
rcv ack1
send pkt0
pkt0
rcv pkt0
send ack0
ack0

day/date

3) ACK loss:

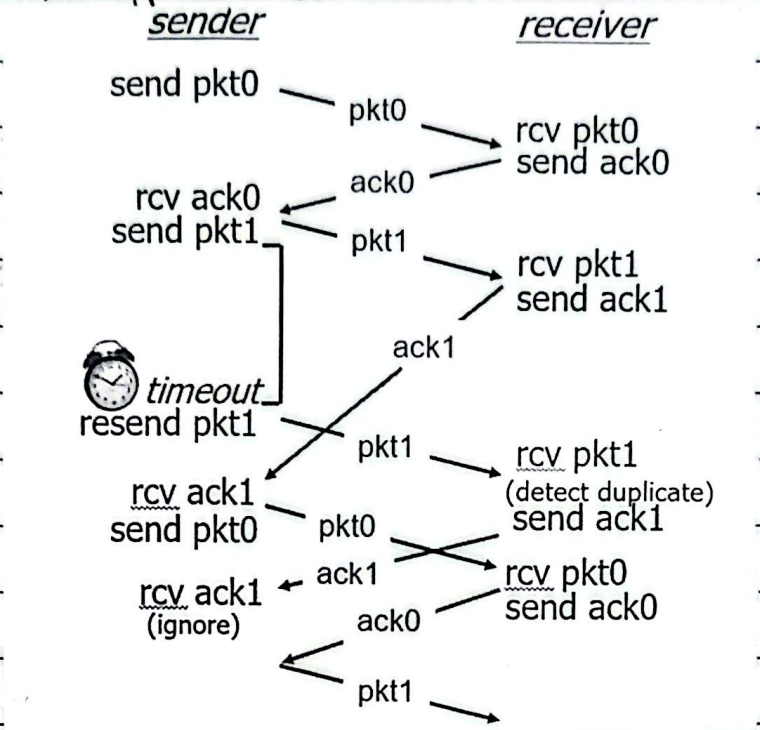


BHATTI

CamScanner

lay/date

4) premature / long RTT scenerion timeout.



Even though this is a problem & few extra are sent once or more, the channel eventually becomes correct again.

BHATTI

CamScanner

day/date

Performance of rdt 3.0:

now rdt 3.0 also follow stop & wait scenario

To evaluate performance of rdt 3.0 or any transfer we use Sender utilization (Usender).

Usender: Fraction of time the sender is actively transmitting data bits onto physical links compared to sitting idle.

Because rdt 3.0 is stop & wait, sender must sit completely idle after sending data till receiving feed back.

BHATTI

CamScanner

day/date

$$U_{sender} = \frac{d_{trans}}{d_{trans} + RTT}$$

$$d_{trans} = \frac{L}{R}$$

L = packet size in bits

R = transmission rate in bits/second.

RTT = Round trip time

Example:

(a) R = 1 Gbps, 15 ms propagation delay & L = 8000 bits.

$$d_{trans} = \frac{8000}{10^9} = 8 \times 10^{-6} \text{ second.}$$

RTT = 2 × Prop delay = 30 ms.

BHATTI

CamScanner

day/date

$$U_{sender} = \frac{0.008 \text{ ms}}{30.008 \text{ ms}}$$

$$= 0.00027$$

so this shows our sender is being equal to no utilization.

which is rdt 3.0 performance is extremely low.

How to increase it?

for this we use Pipelining.

BHATTI

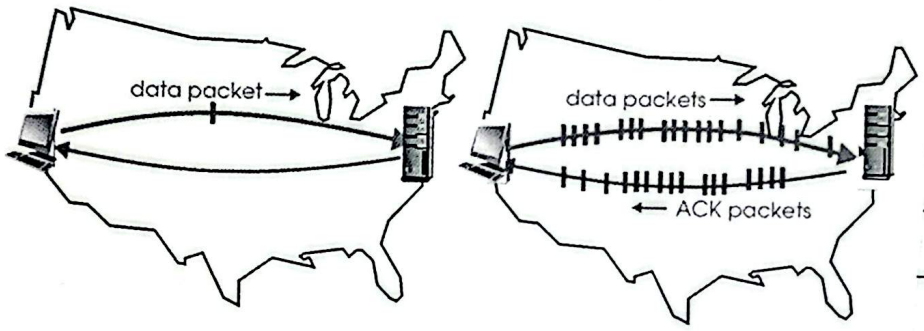
CamScanner

day/date

rdt 3.0 : pipelined protocols:

now we will use rdt 3.0 but with pipelining:

Pipelinging: Instead of sending one packet at a time, send multiple, i.e "yet to be acknowledged packet's"



(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

For this:

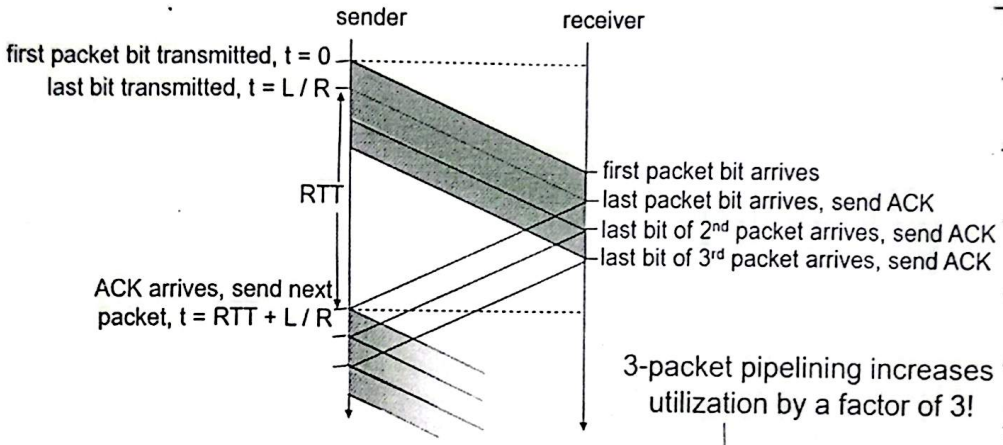
- range of sequence numbers must be increased.
- buffering at sender or receiver.
  - storing multiple packets temporily.

BHATTI

CamScanner

Consider the same performance example we did in normal rdt 3.0 but consider now we send 3 packets at a time:

Pipelining: increased utilization



3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L/R}{RTT + L/R} = \frac{.0024}{30.008} = 0.00081$$

Two major pipelined protocols:

1) Go-Back-N protocol
2) Selective Repeat (SR) protocol

BHATTI

CamScanner

/date

1) Go Back -N protocol:

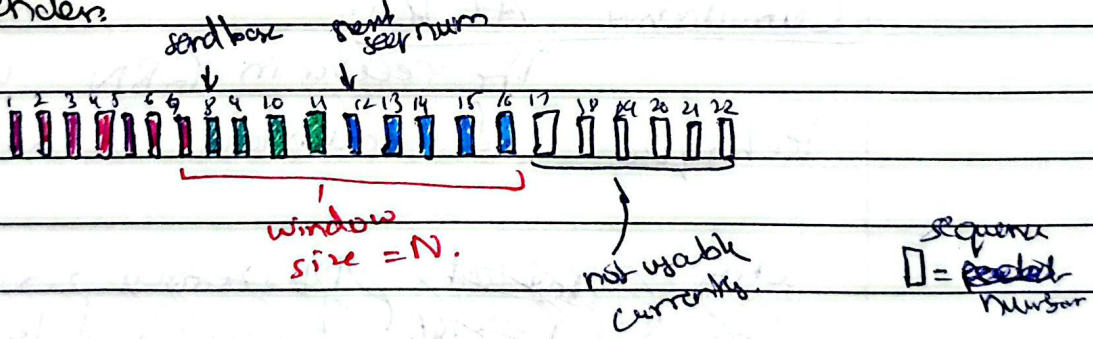
This allows sender to pipeline & send multiple packets at once.

Maximum size = Window size = N.

without waiting for an immediate acknowledgement.

However even if one early packet is lost or timeout occur all next packets need to be retransmitted — Hence the name: Go Back - N.

In sender:



- already gone & acknowledged.
- sent but not yet acknowledged.
- not sent but usable

BHATTI

y/date

The four regions:

[0, send base − 1] : Already acknowledged.

[send_base, nextseqnum − 1] : Sent not yet acknowledged. (in flight)

[nextseqnum, send_base + N − 1] : ~~Already sent~~ next seq num. Sequence number available to send.

[send_base + N, ∞] : sequence numbers blocked until window slides forward.

Cumulative ACK's:

The receiver in GBN uses this technique & maintains a very simple state.

~~When [illegible] correctly at its~~

This technique allows receiver to only keep ACK of oldest sequence number upto n.

BHATTI

CamScanner

//date

- When sender receives the ACK(n) then it moves the window forward, starting at n+1.

Timer now works for the oldest in-flight packet.

e.g

Sequence 1, 2, 3, 4 were sent for one ACK was received. but now 2 becomes oldest in flight. If we don't receive its ACK, retransmit all 2, 3, 4., at time out.

BHATTI

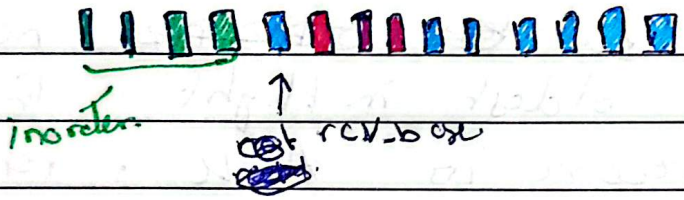
CamScanner

day/date

GBN receiver

sends ~~ACK~~ ACK for correctly-received packet so far, with highest in-order seq #.

Receiver view:



- not received.
- received but cannot acknowledge yet.
- received and acknowledged.

rcv_base: This is the number that is incremented when we receive the particular packet. If the particular sequence number doesn't arrive & ahead of its arrive they are discarded and are NOT ACK until rcv_base number's packet comes

BHATTI

ay/date

Out of order Packet:

Like the sequence i made on previous page. If something like that happens then all the next packet that did came but were not acknowledged are discarded.

- re-ACK pkt with highest in-order sequence number which is (rcv.base - 1). when out-of-order packet comes.

GBN Timer (Imp)

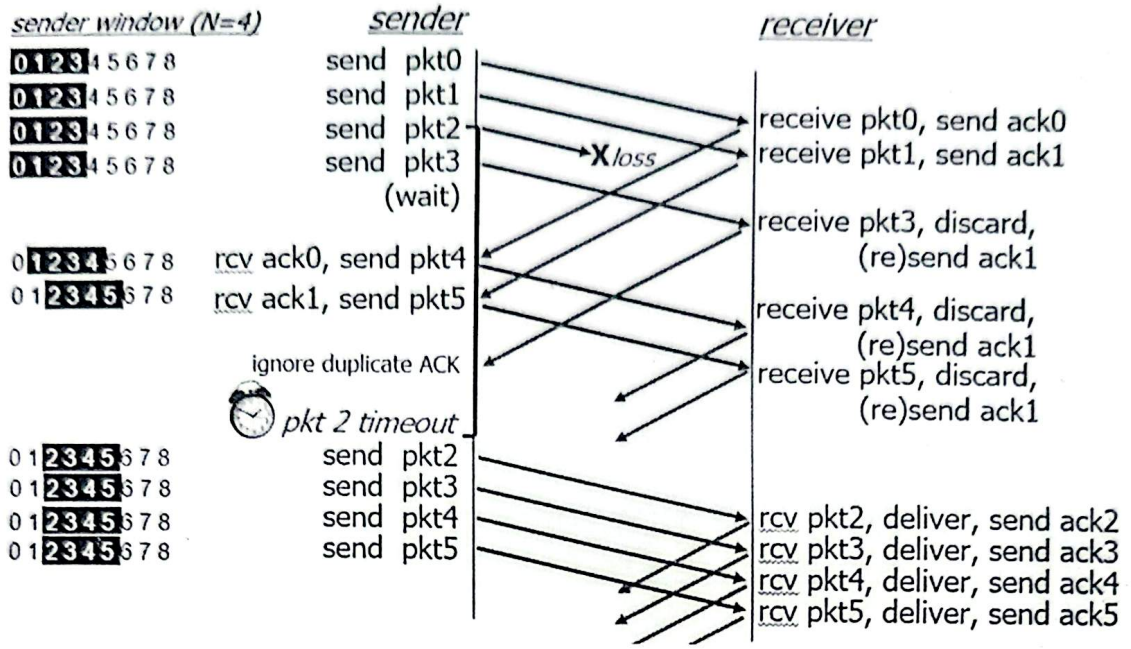
Consider the send base is at sequence 0 & we are sending pkt 0, 1, 2, 3 (window size is 4). In this pkt 1 is lost.

In this timer runs for pkt 0 when its ACK comes, timer restarts so Now even if we had send pkt 1 10 seconds before, it will still take us full timeout to retransmit it.

* Timer starts before transmission. (So transmission delay comes after timer & is included in timer time).

BHATTI

Go-Back-N in action



GBN seems another example.

Assume window size N = 4,

sender sends : Pkt 0, Pkt 1, Pkt 2, Pkt 3.

Pkt 2 is lost in network.

Receiver gets Pkt 0 → sends ACK 0.

" gets Pkt 1 → sends ACK 1

now our rcv_base = 2.

Receiver gets Pkt 3 but rcv_base = 2 so

pk3 is discarded

& it resends ACK 1.

BHATTI

CamScanner

/date

Then time out occurs in pkt2 at sender. Sender goes-back-N & re sends pkt2 & pkt3 (All-in flight packages)

Limitations of GBN:

- When network latency or window sizes are large, dropping a single packet forces the sender to retransmit a vast number of packets that were actually received but discarded.

To save this we have Selected Repeat.

Size of window & Total amount of Seq numbers?

At minimum:

window size = sequence numbers - 1

or

window size + 1 = sequence number.

BHATTI

CamScanner

Why minimum N+1 = sequence numbers needed?

consider Scenario N = 4 & 4 sequence number : 0, 1, 2, 3

now a sender sends 4 packets ~~lost~~ ~~ACK0~~ is ~~lost~~ and expect ACK0, ACK1, ACK2, ACK3 but consider all ACK lost. (all ACK0 – ACK3 lost).

- now receiver is expecting the new packet with sequence 0 but sender's ACK0 for old packet times out & it sends the old packet 0. The receiver consider this as new packet 0 but it is a duplicate

So DATA Corruption.

But if N = 4 & sequence number 0, 1, 2, 3, 4
(5)

This wouldn't happen. because receiver would be waiting for 4 packet sequence 4 & even if sender sends old packet sequence 0 it will be discarded as it is duplicate.

lay/date

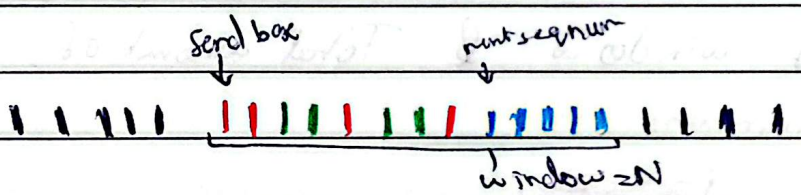
2) Selective Repeat:

In this receiver individually acknowledges all correctly received ~~packages~~ packets.

Receiver has a buffer, in which it can store out-of-order packets for eventual in order delivery.

- Sender in this maintains timer for all UnACKed Packets. (not only the oldest but all) & also maintains a buffer too for all ack'ed Packages.

- Now in this sender might get a view a like:



1 not usable
1 already ack'ed
1 sent but not ack'ed
1 usable but not sent.

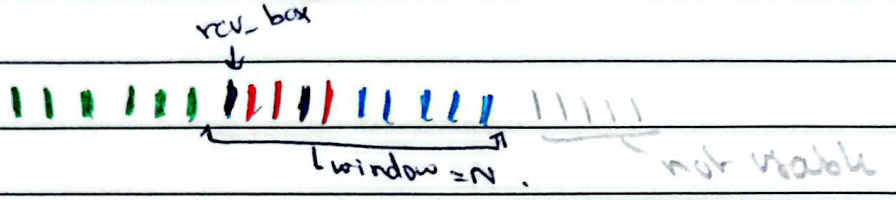
* In this diagram we can see that some not acked exist before already acked because of out of order delivery but they are buffered so no retransmission

BHATTI

CamScanner

day/date

In receiver's view:



1 = acceptable (within window).
1 = out of order but buffered & ack'ed
1 = Expected but not received
1 = already ack'ed & received. (in order).

* now receiver also gets a window

rcv_base: Sequence number of lowest expected in order delivery.

Buffering: If an out of order packet n arrives, the receiver stores it in a buffer & immediately sends ACK n.

In-order delivery: When packet rcv_base arrives, the receiver delivers it plus any consecutively buffered packets to the app layer at once.

BHATTI

CamScanner

Selective repeat: sender and receiver

sender

data from above:

- if next available seq # in window, send packet

timeout(*n*):

- resend packet *n*, restart timer

ACK(*n*) in [sendbase,sendbase+N]:

- mark packet *n* as received
- if n smallest unACKed packet, advance window base to next unACKed seq #

receiver

packet *n* in [rcvbase, rcvbase+N-1]

- send ACK(*n*)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order packets), advance window to next not-yet-received packet

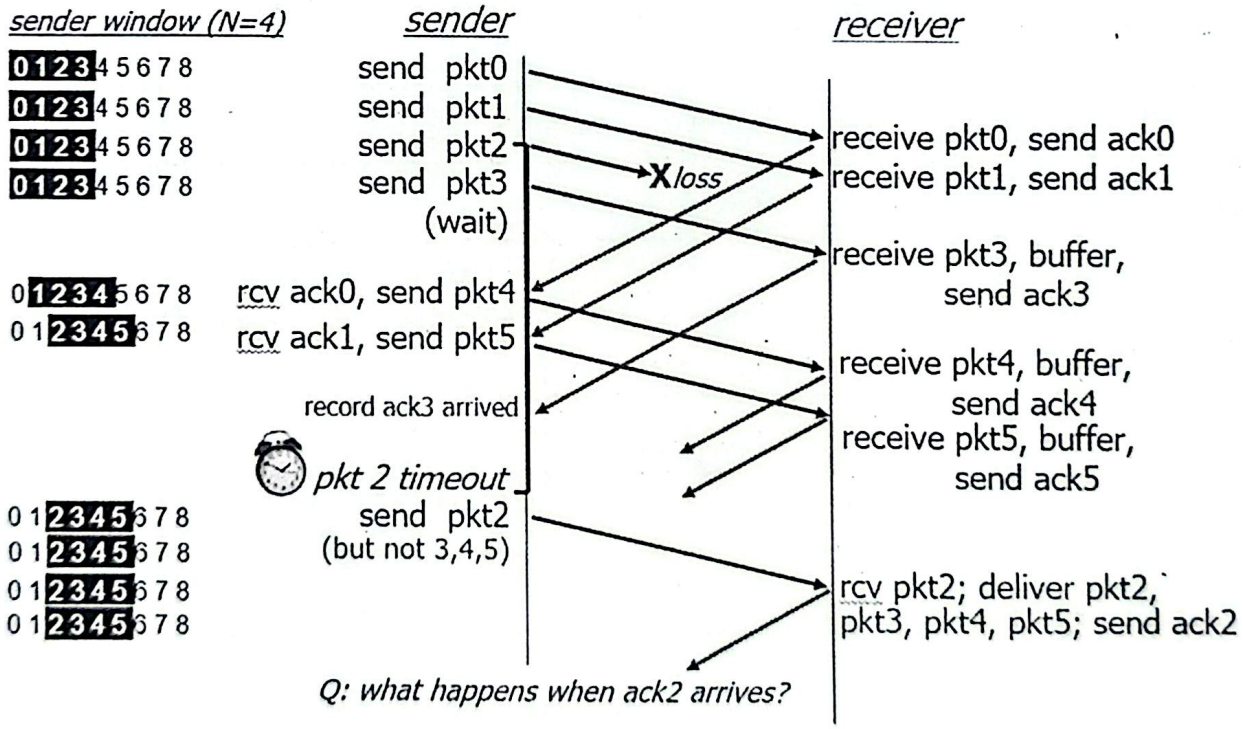
packet *n* in [rcvbase-N,rcvbase-1]

- ACK(*n*)

otherwise:

- ignore

Selective Repeat in action



CamScanner

/date

Minimum Sequence Space ?

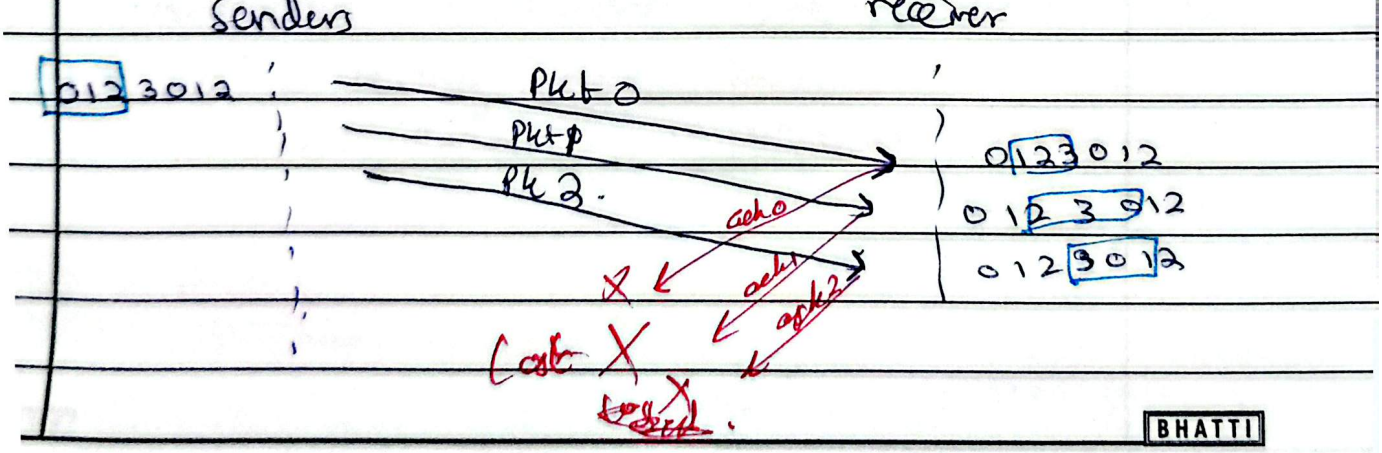
Its minimum sequence numbers = 2N
= 2* window size

This is because now we need to cater for both windows

we need to prevent overlap between receiver window & sender window . so sequence space must be twice size of window

Understanding through example

e.g lets take sequence Number 2 bit i.e 0, 1, 2, 3 & window N = 3



y/date

now because they are lost but our sender window still on [0 1 2] so when timer runs out it sends 0 again. But see receiver window is [3 0 1]. it has space for 0. when 0 comes it is buffered. but this is old 0 pkt which is a duplicate but receiver doesn't know so error.

This is why our window size if N then sequence numbers should be at least 2N.

* But this is a naive (Jugaad) solution.

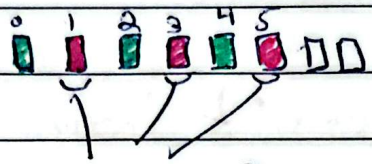
BHATTI

CamScanner

ay/date

Imp:

* A window in SR can be like:
  e.g N = 3



These 3 are in window.
so a window doesn't need to be continuous here

* Sender also has a buffer. This is how it know which packets are already sent & ack'ed

* Never 2 packets will go at same time, they can go after one another by a delay of transmission delay.
Because even if pipelined remember, there is only one wire so only one packet can be transmitted at a time.
& then 2nd, then 3rd etc.
Pipeline means it doesn't have to wait for feedback to send next packet.

BHATTI

CamScanner

day/date

* Timer starts before transmission

* Every pkt has its own timer. (Not one timer like GBN).

* This doesn't have Cumulative ACK at all as both windows have own base.
So if a ACK is lost that Packet will retransmit. & so until sender recieves an ACK of the pkt.

BHATTI

CamScanner